

SOC LAB 6

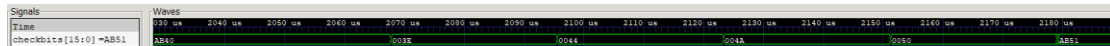
Workload optimized SOC – baseline

312611096 黃鈺淳 / 511506022 何佳玲 / 312605003 王語

2023.12.14

➤ Waveform

1. Matrix Multiplication



$$A = \begin{bmatrix} 0 & 1 & 2 & 3 \\ 0 & 1 & 2 & 3 \\ 0 & 1 & 2 & 3 \\ 0 & 1 & 2 & 3 \end{bmatrix} \quad B = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix}$$
$$A*B = \begin{bmatrix} 62 & 68 & 74 & 80 \\ 62 & 68 & 74 & 80 \\ 62 & 68 & 74 & 80 \\ 62 & 68 & 74 & 80 \end{bmatrix}$$

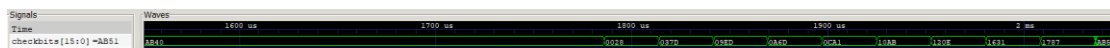
可以看到波形 mprj_io[31:16]依序收到

1. 0xAB40 // Start Flag
2. 0x003E // 62 = A*B[0][0]
3. 0x0044 // 68 = A*B[0][1]
4. 0x004A // 74 = A*B[0][2]
5. 0x0050 // 80 = A*B[0][3]
6. 0xAB51 // End Flag

2. Quick Sort

A = {893, 40, 3233, 4267, 2669, 2541, 9073, 6023, 5681, 4622}

A_sorted = { 40, 893, 2541, 2669, 3233, 4267, 4622, 5681, 6023, 9073}

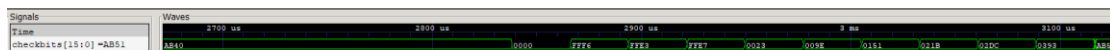


可以看到波形 mprj_io[31:16]依序收到

1. 0xAB40 // Start Flag
2. 0x0028 // 40 = A_sorted[0]
3. 0x37D // 893 = A_sorted[1]

4. 0x09ED // 2541 = A_sorted[2]
5. 0xA6D // 2669 = A_sorted[3]
6. 0xCA1 // 3233 = A_sorted[4]
7. 0x10AB // 4267 = A_sorted[5]
8. 0x120E // 4622 = A_sorted[6]
9. 0x1631 // 5681 = A_sorted[7]
10. 0x1787 // 6023 = A_sorted[8]
11. 0xAB51 // End Flag

3. FIR



tap = {0,-10,-9,23,56,63,56,23,-9,-10,0}

input = {1,2,3,4,5,6,7,8,9,10,11}

FIR = tap[i] * input[n - i]

可以看到波形圖 mprj_io[31:16]依序收到

1. 0xAB40 // Start Flag
2. 0x0000 // $0 = 0 \times 1$
3. 0xFFFF6 // $-10 = 0 \times 2 + (-10) \times 1$
4. 0xFFE3 // $-29 = 0 \times 3 + (-10) \times 2 + (-9) \times 1$
5. 0xFFE7 // $-25 = 0 \times 4 + (-10) \times 3 + (-9) \times 2 + 23 \times 1$
6. 0x0023 // $35 = 0 \times 5 + (-10) \times 4 + (-9) \times 3 + 23 \times 2 + 56 \times 1$
7. 0x009E // $158 = 0 \times 6 + (-10) \times 5 + (-9) \times 4 + 23 \times 3 + 56 \times 2 + 63 \times 1$
8. 0x0151 // $337 = 0 \times 7 + (-10) \times 6 + (-9) \times 5 + 23 \times 4 + 56 \times 3 + 63 \times 2 + 56 \times 1$
9. 0x021B // $539 = 0 \times 8 + (-10) \times 7 + (-9) \times 6 + 23 \times 5 + 56 \times 4 + 63 \times 3 + 56 \times 2 + 23 \times 1$
10. 0x02DC // $732 = 0 \times 9 + (-10) \times 8 + (-9) \times 7 + 23 \times 6 + 56 \times 5 + 63 \times 4 + 56 \times 3 + 23 \times 2 + (-9) \times 1$
11. 0x0393 // $915 = 0 \times 10 + (-10) \times 9 + (-9) \times 8 + 23 \times 7 + 56 \times 6 + 63 \times 5 + 56 \times 4 + 23 \times 3 + (-9) \times 2 + (-10) \times 1$
12. 0xAB51 // End Flag

4. UART

tx_data = 61

LA Test 1 started

tx data bit index 0: 1

tx data bit index 1: 0

tx data bit index 2: 1

tx data bit index 3: 1

tx data bit index 4: 1

tx data bit index 5: 1

tx data bit index 6: 0

tx data bit index 7: 0

➤ **Note book output**

```
Start Caravel Soc
Waiting for interrupt
ab 11 Matmul – start flag
00 3e matmul[0]
00 44 matmul[1]
00 4a matmul[2]
00 50 matmul[3]
ab 19 Matmul – end flag
ab 21 Qsort – start flag
00 28 A_sorted[0]
03 7d A_sorted[1]
09 ed A_sorted[2]

6d
0c a1 A_sorted[4]
10 ab A_sorted[5]
12 0e A_sorted[6]
16 31 A_sorted[7]
17 87 A_sorted[8]
23 71 A_sorted[9]
ab 29 Qsort – end flag
ab 31 FIR – start flag
00 00 FIR_output[0]
ff f6 FIR_output[1]
ff e3 FIR_output[2]
ff e7 FIR_output[3]
00 23 FIR_output[4]
00 9e FIR_output[5]
01 51 FIR_output[6]
02 1b FIR_output[7]
02 dc FIR_output[8]
03 93 FIR_output[9]
04 4a
ab 39 FIR – end flag
44 6f 6e 65 20 48 6f 6d 65 77 6f 72 6b UART – Done Homework
main(): uart_rx is cancelled now
```

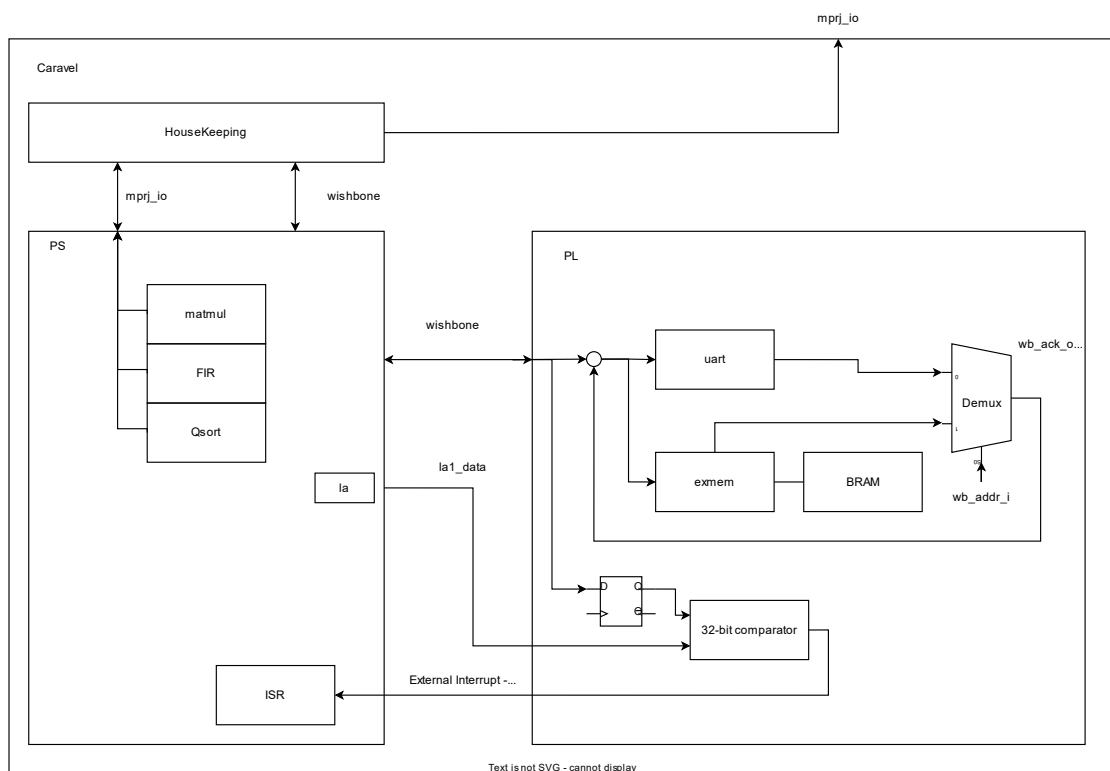
Martix Multiplication	
Start Flag	0xAB11
End Flag	0xAB19
Qsort	
Start Flag	0xAB21
End Flag	0xAB29
FIR	
Start Flag	0xAB31
End Flag	0xAB39

➤ How do you verify your answer from notebook

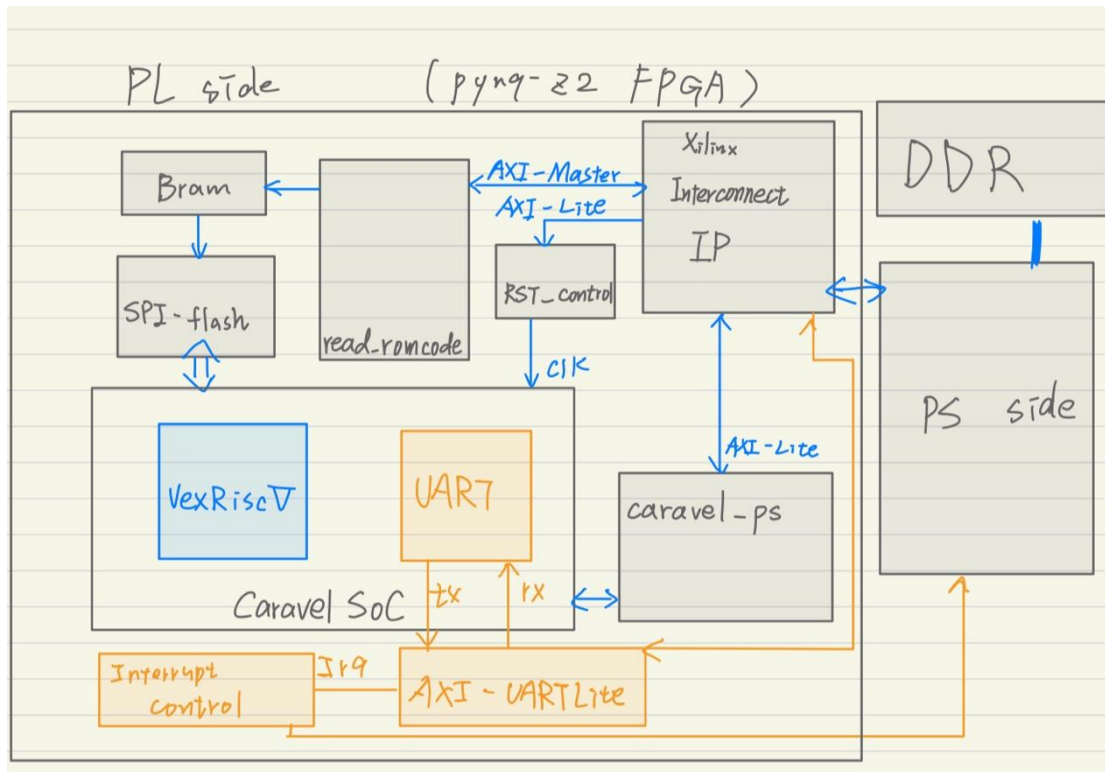
當 wishbone 傳送資料至 mprj 時，資料也會經過 PL 端，可以 latch 住資料並

與 logic analyzer 資料做比對，若此時資料不同，則為資料更新，就發送

IRQ1 通知 PS 發送 UART 資料。



➤ Block design



➤ Timing report/ resource report after synthesis

1	Min Delay Path:				
2					
3	Block (PBT):	0.0000	0.0000	0.0000	0.0000
4	Source:				
5	Destination:				
6	Path:				
7	Path Delay:				
8	Path Delay:				
9	Path Delay:				
10	Path Delay:				
11	Path Delay:				
12	Path Delay:				
13	Path Delay:				
14	Path Delay:				
15	Path Delay:				
16	Path Delay:				
17	Path Delay:				
18	Path Delay:				
19	Path Delay:				
20	Path Delay:				
21	Path Delay:				
22	Path Delay:				
23	Path Delay:				
24	Path Delay:				
25	Path Delay:				
26	Path Delay:				
27	Path Delay:				
28	Path Delay:				
29	Path Delay:				
30	Path Delay:				
31	Path Delay:				
32	Path Delay:				
33	Path Delay:				
34	Path Delay:				
35	Path Delay:				
36	Path Delay:				
37	Path Delay:				
38	Path Delay:				
39	Path Delay:				
40	Path Delay:				
41	Path Delay:				
42	Path Delay:				
43	Path Delay:				
44	Path Delay:				
45	Path Delay:				
46	Path Delay:				
47	Path Delay:				
48	Path Delay:				
49	Path Delay:				
50	Path Delay:				
51	Path Delay:				
52	Path Delay:				
53	Path Delay:				
54	Path Delay:				
55	Path Delay:				
56	Path Delay:				
57	Path Delay:				
58	Path Delay:				
59	Path Delay:				
60	Path Delay:				
61	Path Delay:				
62	Path Delay:				
63	Path Delay:				
64	Path Delay:				
65	Path Delay:				
66	Path Delay:				
67	Path Delay:				
68	Path Delay:				
69	Path Delay:				
70	Path Delay:				
71	Path Delay:				
72	Path Delay:				
73	Path Delay:				
74	Path Delay:				
75	Path Delay:				
76	Path Delay:				
77	Path Delay:				
78	Path Delay:				
79	Path Delay:				
80	Path Delay:				
81	Path Delay:				
82	Path Delay:				
83	Path Delay:				
84	Path Delay:				
85	Path Delay:				
86	Path Delay:				
87	Path Delay:				
88	Path Delay:				
89	Path Delay:				
90	Path Delay:				
91	Path Delay:				
92	Path Delay:				
93	Path Delay:				
94	Path Delay:				
95	Path Delay:				
96	Path Delay:				
97	Path Delay:				
98	Path Delay:				
99	Path Delay:				
100	Path Delay:				

```

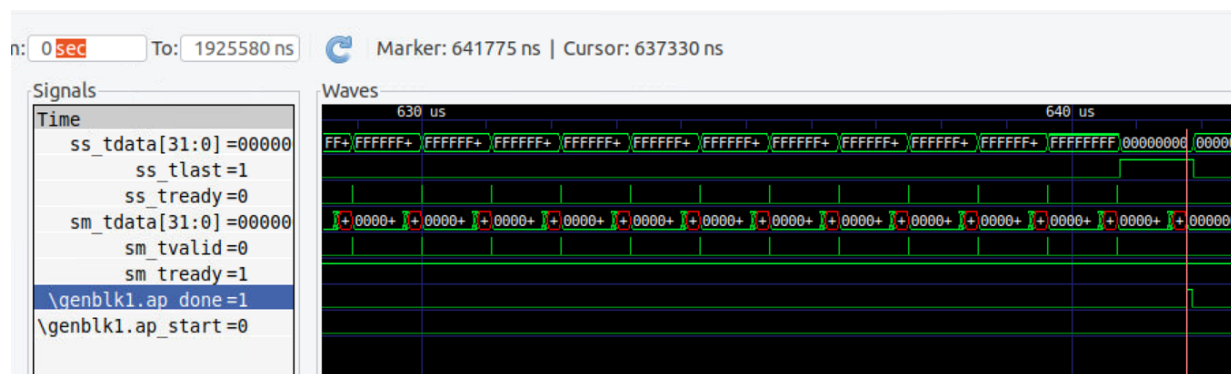
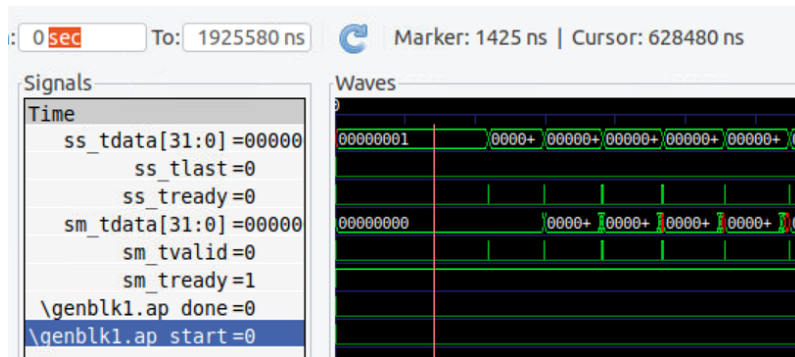
1 Max Delay Paths
2 -----
3 Slack (MET) : 16.696ns (required time - arrival time)
4 Source: design_1_i/output_pin_0/inst/control_s_axi_U/int_outpin_ctrl_reg[0]/C
5 (rising edge-triggered cell FDRE clocked by clk_fpga_0 {rise@0.000ns fall@12.500ns period=25.000ns})
6 Destination: design_1_i/caravel_0/inst/housekeeping/serial_data_staging_2_reg[0]/CLR
7 (recovery check against rising-edge clock clk_fpga_0 {rise@0.000ns fall@12.500ns period=25.000ns})
8 Path Group: **async_default**
9 Path Type: Recovery (Max at Slow Process Corner)
10 Requirement: 25.000ns (clk_fpga_0 rise@25.000ns - clk_fpga_0 rise@0.000ns)
11 Data Path Delay: 7.167ns (logic 0.580ns (8.093%) route 6.587ns (91.907%))
12 Logic Levels: 1 (LUT1-1)
13 Clock Path Skew: -0.355ns (DCD - SCD + CPR)
14 Destination Clock Delay (DCD): 2.653ns = ( 27.653 - 25.000 )
15 Source Clock Delay (SCD): 3.137ns
16 Clock Pessimism Removal (CPR): 0.129ns
17 Clock Uncertainty: 0.377ns ((TSJ^2 + TIJ^2)^1/2 + DJ) / 2 + PE
18 Total System Jitter (TSJ): 0.071ns
19 Total Input Jitter (TIJ): 0.750ns
20 Discrete Jitter (DJ): 0.000ns
21 Phase Error (PE): 0.000ns
22
23 Location Delay type Incr(ns) Path(ns) Netlist Resource(s)
24 -----
25 (clock clk_fpga_0 rise edge)
26
27 PS7_X0Y0 PS7 0.000 0.000 r design_1_i/processing_system7_0/inst/PS7_i/FCLKCLK[0]
28 net (fo=1, routed) 1.193 1.193 design_1_i/processing_system7_0/inst/FCLK_CLK_unbuffered[0]
29 BUFGCTRL_X0Y20 BUFG (Prop_bufg_I_O) 0.101 1.294 r design_1_i/processing_system7_0/inst/buffer_fclk_clk_0.FCLK_CLK_0_BUFG/O
30 net (fo=5307, routed) 1.843 3.137 design_1_i/output_pin_0/inst/control_s_axi_U/ap_clk
31 SLICE_X43Y103 FDRE r design_1_i/output_pin_0/inst/control_s_axi_U/int_outpin_ctrl_reg[0]/C
32 -----
33 SLICE_X43Y103 FDRE (Prop_fdre_C_Q) 0.456 3.593 r design_1_i/output_pin_0/inst/control_s_axi_U/int_outpin_ctrl_reg[0]/Q
34 net (fo=9, routed) 1.746 5.339 design_1_i/caravel_0/inst/housekeeping/resetb
35 SLICE_X45Y72 LUT1 (Prop_lut1_I_O) 0.124 5.463 f design_1_i/caravel_0/inst/housekeeping/FSM_sequential_xfer_state[0]_i_2/0
36 net (fo=662, routed) 4.841 10.304 design_1_i/caravel_0/inst/housekeeping/pad_flash_csb_oeb0
37 SLICE_X47Y87 FDCE f design_1_i/caravel_0/inst/housekeeping/serial_data_staging_2_reg[0]/CLR
38 -----
39
40 (clock clk_fpga_0 rise edge)
41
42 PS7_X0Y0 PS7 25.000 25.000 r design_1_i/processing_system7_0/inst/PS7_i/FCLKCLK[0]
43 net (fo=1, routed) 1.088 26.088 design_1_i/processing_system7_0/inst/FCLK_CLK_unbuffered[0]
44 BUFGCTRL_X0Y20 BUFG (Prop_bufg_I_O) 0.091 26.179 r design_1_i/processing_system7_0/inst/buffer_fclk_clk_0.FCLK_CLK_0_BUFG/O
45 net (fo=5307, routed) 1.474 27.653 design_1_i/caravel_0/inst/housekeeping/clock
46 SLICE_X47Y87 FDCE r design_1_i/caravel_0/inst/housekeeping/serial_data_staging_2_reg[0]/C
47 clock pessimism 0.129 27.782
48 clock uncertainty -0.377 27.405
49 SLICE_X47Y87 FDCE (Recov_fdce_C_CLR) -0.405 27.000 design_1_i/caravel_0/inst/housekeeping/serial_data_staging_2_reg[0]
50 -----
51 required time 27.000
52 arrival time -10.304
53 -----
54 slack 16.696

```

- Latency for a character loop back using UART
- Suggestion for improving latency for UART loop back
- What else do you observe

- What is the FIR engine theoretical throughput, i.e. data rate? Actually measured throughput? data rate? Actually measured throughput?

Ap_start assert at 1425 ns ,and Ap_done assert at 641775 ns



以 Lab3 結果而言，最理想的情況下可以在 640350 ns 內完成 600 筆

資料的運算（每筆資料相當於 11 次乘法，10 次加法）。

Data rate = 9370000

Throughput = 196,770,000

```
----- Times# 1 -----
Start latency-timer
Stop latency-timer
Elapsed time:      959150 [ns]
-----
Correct, ans = 118, golden_ans = 118

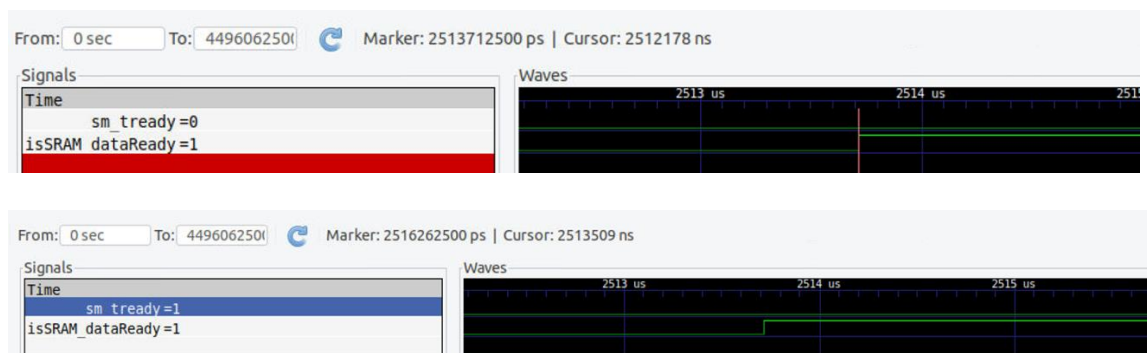
----- Times# 2 -----
Start latency-timer
Stop latency-timer
Elapsed time:      970900 [ns]
-----
Correct, ans = 118, golden_ans = 118

----- Times# 3 -----
Start latency-timer
Stop latency-timer
Elapsed time:      970900 [ns]
-----
Correct, ans = 118, golden_ans = 118
```

但是實際上量測到的結果，在 970900 ns 內只能完成 64 筆資料的運算

Data rate = 61800
Throughput = 1,297,800

- What is latency for firmware to feed data?



Latency : 2,550 ns

- What techniques used to improve the throughput?

由於目前效能受限於 Firmware，可以嘗試以下方法提升 throughput

1. 增加資料傳輸效率，以本次實驗為例，單筆 $x[n]$ 不會用到 32bits，所以可以合併多筆 $x[n]$ 一次傳送，假設 $x[n]$ 的範圍為 1~64，代表只需要使用到 7bits，可以讓 $data[31:25]$ 代表 $x[n]$ ， $data[24:18]$ 代表 $x[n+1]$ ，以此類推， $Y[n]$ 的部分也可以仿效。
2. 增加 Input/Output Buffer，如此一來可以配合 CPU 節奏收發資料。
3. 加快 Caraval SoC 運作頻率

- Does bram12 give better performance, in what way?

多出更多 RAM 能讓運算更有彈性，減少 control 負擔，但在本次實驗中成效有限，因為硬體做再快也會受限於韌體速度。

Bram12 多了一個記憶體位置，可以當作 Buffer 使用，也就是說當 FIR 還未完成當前運算的期間，CPU 就能先將下一次運算需要的 $x[n]$ 寫入，並且不會覆蓋掉可能需要用到的資料。

- Can you suggest other method to improve the performance?

進一步推廣單次傳送多筆資料的想法，並且推廣至更廣泛的數值，譬如使用 BF16 格式進行資料傳輸，單次傳輸可以傳送 2 筆資料，而且數值範圍廣泛，雖然增加硬體負擔，需要多進行一個解碼、編碼的任務，但以上百個 cycle 內完成的前提下硬體依舊游刃有餘。

使用 Share memory 進行資料傳輸，CPU 不必等待 FIR 接收資料，可以事先將多筆 $x[n]$ 資料儲存在共用記憶體，等到 FIR 需要時再讀取，反之 FIR 也能將運算完成的 $Y[n]$ 存放在共享記憶體，等 CPU 有空再去讀取。

- Any other insights ?

對比將 Firmware 存放在 SPI Flash 以及 User RAM 之運行時間

Initial :8063625ns

```
λ make
Reading counter_la_fir.hex
counter_la_fir.hex loaded into memory
Memory 5 bytes = 0x6f 0x00 0x00 0x0b 0x13
VCD info: dumpfile counter_la_fir.vcd opened for output.
Start latency-timer
Stop latency-timer
Elapsed time:      8063625 [ns]
counter_la_fir_tb.v:196: $finish called at 9050387500 (1ps)
```

Improved:921772ns

```
ubuntu@ubuntu2004:~/lab4_2/lab4-2better/testbench/counter_la_fir$ make
Reading counter_la_fir.hex
counter_la_fir.hex loaded into memory
Memory 5 bytes = 0x6f 0x00 0x00 0x0b 0x13
VCD info: dumpfile 4-2.vcd opened for output.
Start latency-timer
Stop latency-timer
Elapsed time:      921775 [ns]
```

控制 feedback 的時間，如下圖

```
always @(posedge clk) begin
    if (rst) begin
        ready <= 1'b0;
        delayed_count <= 16'b0;
    end else begin
        ready <= 1'b0;
        if (wbs_adr_1[31:16]==16'h3800) begin
            // $display("0x3800");
            if (valid && !ready) begin
                // $display("delay_count = %d",delayed_count);
                if (delayed_count == DELAYS) begin
                    delayed_count <= 16'b0;
                    ready <= 1'b1;
                    // $display("ack");
                end
            end else begin
                delayed_count <= delayed_count + 1;
            end
        end
    end
end
end
```

➤ TIME REPORT

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 0.530 ns	Worst Hold Slack (WHS): 0.132 ns	Worst Pulse Width Slack (WPS): 0.000 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 416	Total Number of Endpoints: 416	Total Number of Endpoints: 416

All user specified timing constraints are met.

1. Slice Logic

Site Type	Used	Fixed	Prohibited	Available	Util%
Slice LUTs*	380	0	0	53200	0.71
LUT as Logic	380	0	0	53200	0.71
LUT as Memory	0	0	0	17400	0.00
Slice Registers	400	0	0	106400	0.38
Register as Flip Flop	260	0	0	106400	0.24
Register as Latch	140	0	0	106400	0.13
F7 Muxes	0	0	0	26600	0.00
F8 Muxes	0	0	0	13300	0.00

* Warning! The Final LUT count, after physical optimizations and full implementation completed, for a more realistic count.

1.1 Summary of Registers by Type

Total	Clock Enable	Synchronous	Asynchronous
0	-	-	-
0	-	-	Set
0	-	-	Reset
0	-	Set	-
0	-	Reset	-
0	Yes	-	-
7	Yes	-	Set
375	Yes	-	Reset
0	Yes	Set	-
18	Yes	Reset	-

- ▼ Design Sources (2)
 - ▼ Non-module Files (1)
 - counter.v
 - ▼ ● **user_project_wrapper** (user_project_wrapper.v) (2)
 - ▼ ● mprj : user_proj_example (user_project.v) (4)
 - bram_fir_extmem : bram (bram.v)
 - fir_DUT : fir (fir.v)
 - tap_RAM : bram (bram.v)
 - data_RAM : bram (bram.v)
 - ▼ ● counter : user_proj_counter (user_proj_example.counter.v) (1)
 - user_bram : bram (bram.v)
- ▼ Constraints (1)
 - ▼ constrs_1 (1)
 - 📄 system_xdc.xdc (target)
- ▼ Simulation Sources (2)
 - ▼ sim_1 (2)
 - ▼ Non-module Files (1)
 - counter.v
 - ▼ ● **user_project_wrapper** (user_project_wrapper.v) (2)
 - > ● mprj : user_proj_example (user_project.v) (4)
 - > ● counter : user_proj_counter (user_proj_example.counter.v) (1)
- ▼ Utility Sources (1)
 - ▼ utils_1 (1)
 - ▼ Design Checkpoint (1)
 - 🔍 user_proj_counter.dcp